Set up a local Postman MCP server

The local server is based on STDIO transport and is hosted locally on an environment of your choice. STDIO is a lightweight solution that's ideal for integration with editors and tools like Visual Studio Code. Install an MCP-compatible VS Code extension, such as GitHub Copilot, Claude for VS Code, or other AI assistants that support MCP. The local server only supports API key authentication (with a Postman API key or Bearer token).

The local server only supports API key authentication (with a Postman API key or Bearer token).

Use cases

Consider using the local Postman MCP server if:

- You want to power local use cases, such as local API testing.
- You have specific security and network requirements.
- You prefer to build the MCP server from the source code in this repo.
- You're working with internal APIs.

Supported configurations

The local server supports the following tool configurations:

- **Minimal** — (Default) Only includes essential tools for basic Postman operations.
- **Code** — Includes tools for searching public and internal API definitions and generating client code.
- **Full** — Includes all available Postman API tools (100+ tools).

option.

To run the server as a Node application, install Node.js before getting started.

Visual Studio Code

Click the button to install the local Postman MCP server in VS Code:

By default, the server uses **Full** mode. To access **Minimal** mode, remove the `--full` flag from the `mcp.json` configuration file. To access **Code** mode, replace the `--full` flag with the `--code` flag.

Manual configuration

You can manually integrate your MCP server with VS Code to use it with extensions that support MCP. To do this, create a `mcp.json` file in your project and add the following JSON block to it:

```
{
    "servers": {
        "postman": {
            "type": "stdio",
            "command": "npx",
            "args": [
                "@postman/postman-mcp-server",
                "--full", // (optional) Use this flag to enable full mode...
                "--code", // (optional) ...or this flag to enable code mode.
                "--region us" // (optional) Use this flag to specify the Pos
tman API region (us or eu). Defaults to us.
            ],
            "env": {
                "POSTMAN_API_KEY": "${input:postman-api-key}"
            }
        }
```

```
18          {
```

Cursor

Click the button to install the local Postman MCP server in Cursor:

Add to Cursor

By default, the server uses **Full** mode. To access **Minimal** mode, remove the `--full` flag from the `mcp.json` configuration file. To access **Code** mode, replace the `--full` flag with the `--code` flag.

Manual installation

To manually integrate your MCP server with Cursor and VS Code, create a `.vscode/mcp.json` file in your project and add the following JSON block to it:

```
{
    "servers": {
        "postman-api-mcp": {
            "type": "stdio",
            "command": "npx",
            "args": [
                "@postman/postman-mcp-server",
                "--full" // (optional) Use this flag to enable full mode.
                "--code" // (optional) Use this flag to enable code mode.
                "--region us" // (optional) Use this flag to specify the Pos
tman API region (us or eu). Defaults to us.
            ],
            "env": {
                "POSTMAN_API_KEY": "${input:postman-api-key}"
            }
        }
    },
    "inputs": [
        {
```

To integrate the local Postman MCP server with Claude, check the latest Postman MCP server release and get the `.mcpb` file:

- Minimal — `postman-api-mcp-minimal.mcpb`
- Code — `postman-mcp-server-code.mcpb`
- Full — `postman-api-mcp-full.mcpb`

For more information, see the Claude Desktop Extensions documentation.

Claude Code

To install the MCP server in Claude Code, run the following command in your terminal:

Minimal

```
$ claude mcp add postman --env POSTMAN_API_KEY=<POSTMAN_API_KEY> -- npx @postma
  n/postman-mcp-server@latest
```

Code

```
$ claude mcp add postman --env POSTMAN_API_KEY=<POSTMAN_API_KEY> -- npx @postma
  n/postman-mcp-server@latest --code
```

Full

```
$ claude mcp add postman --env POSTMAN_API_KEY=<POSTMAN_API_KEY> -- npx @postma
  n/postman-mcp-server@latest --full
```

Codex

To install the local server, use the API key installation method. Set the `POSTMAN_API_KEY` environment variable and invoke the MCP server using `npx`.

Minimal

Code

```
$ codex mcp add postman --env POSTMAN_API_KEY=<POSTMAN_API_KEY> -- npx @postma
  n/postman-mcp-server --code
```

Full

```
$ codex mcp add postman --env POSTMAN_API_KEY=<POSTMAN_API_KEY> -- npx @postma
  n/postman-mcp-server --full
```

Windsurf

To manually install the MCP server in Windsurf, do the following:

1. Click **Open MCP Marketplace** in Windsurf.
2. Enter “Postman” in the search text box to filter the marketplace results.
3. Click **Install**.
4. When prompted, enter a valid Postman API key.
5. Select the tools that you want to enable, or click **All Tools** to select all available tools.
6. Turn on **Enabled** to enable the Postman MCP server.

Manual installation

Copy the following JSON config into the `.codeium/windsurf/mcp_config.json` file:

```
{
    "mcpServers": {
        "postman": {
            "args": [
                "@postman/postman-mcp-server"
            ],
            "command": "npx",
            "disabled": false,
```

```
            "POSTMAN_API_KEY": "<POSTMAN_API_KEY>"
        }
    }
}
```

Antigravity

To install the MCP server in Antigravity, click **Manage MCP servers > View raw config**. Then, copy the following JSON config into the `mcp_config.json` file:

```
{
    "mcpServers": {
        "postman": {
            "args": [
                "@postman/postman-mcp-server"
            ],
            "command": "npx",
            "disabled": false,
            "disabledTools": [],
            "env": {
                "POSTMAN_API_KEY": "<POSTMAN_API_KEY>"
            }
        }
    }
}
```

GitHub Copilot CLI

Use the Copilot CLI to interactively add the MCP server:

```
$ /mcp add
```

Manual installation

Copy the following JSON config into the `~/.copilot/mcp-config.json` file:

```
        "postman": {
            "command": "npx",
            "args": [
                "@postman/postman-mcp-server"
            ],
            "env": {
                "POSTMAN_API_KEY": "<POSTMAN_API_KEY>"
            }
        }
    }
}
```

Gemini CLI extension

To install the MCP server as a Gemini CLI extension, run the following command in your terminal:

```
$ gemini extensions install https://github.com/postmanlabs/postman-mcp-server
```

Kiro

To install the local Postman MCP Server in Kiro, click the install button for the version that you want to use:

To install the Postman MCP server with Kiro powers, go to Kiro Powers and navigate to **API Testing with Postman** in the **Browse powers** section. Then, click **Add to Kiro**.

Manual installation

To install the Postman MCP Server manually, do the following:

1. Launch Kiro and click the Kiro ghost icon in the left sidebar.

3. Add the following JSON block to the `mcp.json` configuration file:

```
{
    "mcpServers": {
        "postman": {
        "command": "npx",
            "args": [
                "@postman/postman-mcp-server"
            ],
            "env": {
                "POSTMAN_API_KEY": "<POSTMAN_API_KEY>"
            },
            "disabled": false,
            "autoApprove": [
                "getAuthenticatedUser"
            ]
        }
    }
}
```

Docker

To install the Postman MCP server in Docker, see the Postman MCP server at Docker MCP Hub. Click **+ Add to Docker Desktop** to automatically install it.

To run the Postman MCP server image in Docker, run the following command in your terminal. Docker automatically discovers, downloads, and runs the Postman MCP server image:

```
$ docker run -i -e POSTMAN_API_KEY="<POSTMAN_API_KEY>" mcp/postman
```

Manual installation

To build and run the server in Docker manually, run the `docker build -t postman-api-mcp-stdio .` command. Then, run one of the following commands:

```
$ docker run -i -e POSTMAN_API_KEY="<POSTMAN_API_KEY>" postman-api-mcp-stdio
```

Full

```
$ docker run -i -e POSTMAN_API_KEY="<POSTMAN_API_KEY>" postman-api-mcp-stdio --
  full
```


Previous
Set up a remote Postman MCP server

Next
Tips and best practices for using the Pos...

Integrations

Workspaces

Plans and pricing

eSignature

Financial Services

Payments

Travel

Resources

Postman Docs

Academy

Community

Templates

Intergalactic

Videos

MCP Servers

Legal and Security

Legal Terms Hub

Terms of Service

Postman Product Terms

Security

Website Terms of Use

Company

About

Careers and culture

Contact us

Partner program

Customer stories

Student programs

Press and media

POSTMAN

Download Postman